Lab-16.1

Name: D. Suraj
2303A53006
Batch:46

Lab 16 – Database Design and Queries: Schema Design and SQL Generation

Task 1 – Student Information System Schema

Task:
Design a database schema for a Student Information System and generate queries using AI.

Instructions:
• Tables: Students, Courses, Enrollments.
• Define primary keys, foreign keys, and relationships.

• **Generate queries:**
  o Insert a new student record.
  o Fetch all courses enrolled by a student.
  o Count number of students in each course.

Expected Output:
• SQL CREATE TABLE statements and queries for student– course relationships.

Prompt:

"Design a normalized SQL database schema for a Student Information System with three tables: Students, Courses, and Enrollments. Define primary keys, foreign keys, and relationships. Generate SQL queries to:

1. Insert a new student record.

2. Fetch all courses enrolled by a student using an INNER JOIN.

3. Count the number of students in each course using GROUP BY. Ensure the syntax is compatible with MySQL 8.0."

Goal:

Create a 3NF schema for Students, Courses, and Enrollments.

queries.sql + 44hctxwwq

```sql
-- 1. DATABASE SETUP
CREATE TABLE Students (
    StudentID INT PRIMARY KEY,
    StudentName VARCHAR(100) NOT NULL,
    Email VARCHAR(100) UNIQUE NOT NULL
);

CREATE TABLE Courses (
    CourseID INT PRIMARY KEY,
    CourseName VARCHAR(100) NOT NULL,
    Credits INT
);

CREATE TABLE Enrollments (
    EnrollmentID INT PRIMARY KEY AUTO_INCREMENT,
    StudentID INT,
    CourseID INT,
    EnrollmentDate DATE,
    FOREIGN KEY (StudentID) REFERENCES Students(StudentID) ON DELETE CASCADE,
    FOREIGN KEY (CourseID) REFERENCES Courses(CourseID) ON DELETE CASCADE
);

-- 2. INSERT DATA
INSERT INTO Students VALUES (23001, 'Aditya Kumar', 'aditya.k@university.edu');
INSERT INTO Students VALUES (23002, 'Srinivas Rao', 'srinivas.r@university.edu');

INSERT INTO Courses VALUES (101, 'AI Assisted Coding', 4);
INSERT INTO Courses VALUES (102, 'Database Management', 3);

INSERT INTO Enrollments (StudentID, CourseID, EnrollmentDate) VALUES (23001, 101, '2026-03-25');
INSERT INTO Enrollments (StudentID, CourseID, EnrollmentDate) VALUES (23001, 102, '2026-03-25');
INSERT INTO Enrollments (StudentID, CourseID, EnrollmentDate) VALUES (23002, 101, '2026-03-25');

-- 3. FETCH STUDENT COURSES
SELECT s.StudentName, c.CourseName, c.Credits
FROM Students s
JOIN Enrollments e ON s.StudentID = e.StudentID
JOIN Courses c ON e.CourseID = c.CourseID
WHERE s.StudentID = 23001;

-- 4. COUNT STUDENTS PER COURSE
SELECT c.CourseName, COUNT(e.StudentID) AS Total_Students
FROM Courses c
LEFT JOIN Enrollments e ON c.CourseID = e.CourseID
GROUP BY c.CourseID, c.CourseName;
```

STDIN

Input for the program (Optional)

Output: 151 ms

```
+---------------+----------------------+----------+
| StudentName   | CourseName           | Credits  |
+---------------+----------------------+----------+
| Aditya Kumar  | AI Assisted Coding   |        4 |
| Aditya Kumar  | Database Management  |        3 |
+---------------+----------------------+----------+
+----------------------+----------------+
| CourseName           | Total_Students |
+----------------------+----------------+
| AI Assisted Coding   |              2 |
| Database Management  |              1 |
+----------------------+----------------+
```

Task 2 – Hospital Management Database

Task:

Create schema and queries for a Hospital Management System.

Instructions:
• Tables: Doctors, Patients, Appointments.
• Use AI to define constraints (unique IDs, valid dates).

• **Generate queries:**
  o List all appointments for a specific doctor.
  o Retrieve patient history by patient ID.
  o Count total patients treated by each doctor.

Expected Output:
• Normalized schema and SQL queries with joins.

Prompt:

"Create a SQL schema for a Hospital Management System including Doctors, Patients, and Appointments tables. Use AI to define constraints such as unique IDs and valid dates. Generate queries to:

1. List all appointments for a specific doctor.

2. Retrieve the complete medical history for a patient by their PatientID.

3. Count the total unique patients treated by each doctor using a JOIN and COUNT DISTINCT."

Goal:

Manage appointments with doctor and patient history.

queries.sql

44hctxwwq

```sql
-- 1. SCHEMA DESIGN
CREATE TABLE Doctors (
    DoctorID INT PRIMARY KEY,
    DoctorName VARCHAR(100) NOT NULL,
    Specialization VARCHAR(50)
);

CREATE TABLE Patients (
    PatientID INT PRIMARY KEY,
    PatientName VARCHAR(100) NOT NULL,
    DOB DATE
);

CREATE TABLE Appointments (
    ApptID INT PRIMARY KEY AUTO_INCREMENT,
    DoctorID INT,
    PatientID INT,
    ApptDate DATE NOT NULL,
    Reason VARCHAR(255),
    FOREIGN KEY (DoctorID) REFERENCES Doctors(DoctorID) ON DELETE CASCADE,
    FOREIGN KEY (PatientID) REFERENCES Patients(PatientID) ON DELETE CASCADE
);

-- 2. INSERT SAMPLE DATA
INSERT INTO Doctors VALUES (101, 'Dr. Smith', 'Cardiology');
INSERT INTO Doctors VALUES (102, 'Dr. Adams', 'Pediatrics');

INSERT INTO Patients VALUES (501, 'John Doe', '1985-05-15');
INSERT INTO Patients VALUES (502, 'Jane Wilson', '1992-08-22');

-- Appointments for Dr. Smith (101)
INSERT INTO Appointments (DoctorID, PatientID, ApptDate, Reason) VALUES (101, 501, '2026-03-20', 'Regular Checkup');
INSERT INTO Appointments (DoctorID, PatientID, ApptDate, Reason) VALUES (101, 502, '2026-03-22', 'Heart Rate Monitoring');

-- Appointment for Dr. Adams (102)
INSERT INTO Appointments (DoctorID, PatientID, ApptDate, Reason) VALUES (102, 501, '2026-03-25', 'Flu Symptoms');

-- 3. GENERATE OUTPUT QUERIES

-- Query 1: List all appointments for Dr. Smith (ID: 101)
SELECT a.ApptDate, p.PatientName, a.Reason
FROM Appointments a
JOIN Patients p ON a.PatientID = p.PatientID
WHERE a.DoctorID = 101;

-- Query 2: Retrieve patient history for John Doe (ID: 501)
SELECT a.ApptDate, d.DoctorName, d.Specialization, a.Reason
FROM Appointments a
JOIN Doctors d ON a.DoctorID = d.DoctorID
WHERE a.PatientID = 501
ORDER BY a.ApptDate DESC;

-- Query 3: Count total unique patients treated by each doctor
SELECT d.DoctorName, COUNT(DISTINCT a.PatientID) AS Total_Patients_Treated
FROM Doctors d
LEFT JOIN Appointments a ON d.DoctorID = a.DoctorID
GROUP BY d.DoctorID, d.DoctorName;
```

STDIN

Input for the program (Optional)

Output:

101 ms

```
+------------+-------------+-----------------------+
| ApptDate   | PatientName | Reason                |
+------------+-------------+-----------------------+
| 2026-03-20 | John Doe    | Regular Checkup       |
| 2026-03-22 | Jane Wilson | Heart Rate Monitoring |
+------------+-------------+-----------------------+
+------------+------------+----------------+-----------------+
| ApptDate   | DoctorName | Specialization | Reason          |
+------------+------------+----------------+-----------------+
| 2026-03-25 | Dr. Adams  | Pediatrics     | Flu Symptoms    |
| 2026-03-20 | Dr. Smith  | Cardiology     | Regular Checkup |
+------------+------------+----------------+-----------------+
+------------+------------------------+
| DoctorName | Total_Patients_Treated |
+------------+------------------------+
| Dr. Smith  |                      2 |
| Dr. Adams  |                      1 |
+------------+------------------------+
```

Task 3 – Library Database

Task:
Design schema for a Library Management System.

Instructions:
• Tables: Books, Members, Loans.
• Use AI to suggest indexing strategy for faster queries.

• **Generate queries:**
  o Retrieve all books currently issued.
  o Find overdue books (loan date > 30 days).
  o Count number of books loaned by each member.

Expected Output:
• Schema + SQL queries demonstrating joins and conditions.

Prompt:

"Generate a SQL schema for a Library Management System with Books, Members, and Loans tables. Suggest an indexing strategy for faster queries on loan dates. Write queries to:

1. Retrieve all books currently issued (where return date is NULL).

2. Find overdue books where the loan date is older than 30 days.

3. Count the number of books loaned by each member."

Goal:

Track book loans and identify overdue items.

queries.sql + 44hctxwwq

```sql
-- Schema & Indexing
CREATE TABLE Books (BookID INT PRIMARY KEY, Title VARCHAR(100), Status VARCHAR(20));
CREATE TABLE Members (MemberID INT PRIMARY KEY, Name VARCHAR(100));
CREATE TABLE Loans (
    LoanID INT PRIMARY KEY, BookID INT, MemberID INT, LoanDate DATE,
    FOREIGN KEY (BookID) REFERENCES Books(BookID),
    FOREIGN KEY (MemberID) REFERENCES Members(MemberID)
);
CREATE INDEX idx_loan_date ON Loans(LoanDate);

-- Data Insertion
INSERT INTO Books VALUES (1, 'Database Systems', 'Issued');
INSERT INTO Members VALUES (10, 'Srinivas');
INSERT INTO Loans VALUES (100, 1, 10, '2026-02-01');

-- Output Query (Overdue > 30 days)
SELECT * FROM Loans WHERE DATEDIFF('2026-03-25', LoanDate) > 30;
```

STDIN

Input for the program (Optional)

Output: 138 ms

```
+--------+--------+----------+------------+
| LoanID | BookID | MemberID | LoanDate   |
+--------+--------+----------+------------+
|    100 |      1 |       10 | 2026-02-01 |
+--------+--------+----------+------------+
```

Task 4 – Real-Time Application: Online Shopping Database
Scenario:

Design a database for an E-commerce platform.
Requirements:
• Tables: Users, Products, Orders, OrderDetails.

• Generate queries to:
  o Retrieve all orders by a user.
  o Find the most purchased product.
  o Calculate total revenue in a given month.
• AI should also suggest normalization improvements and query optimization.

Expected Output:
• Complete schema with relationships + SQL queries for analytics.

AI Prompt:
Design an E-commerce database (Users, Products, Orders, OrderDetails) in 3NF. Generate queries for user history, most purchased product, and monthly revenue. Suggest query optimization.

Goal: E-commerce analytics and normalization.

queries.sql + 44hctxwwq

```sql
-- Schema Design
CREATE TABLE Users (UserID INT PRIMARY KEY, UserName VARCHAR(100));
CREATE TABLE Products (ProductID INT PRIMARY KEY, ProductName VARCHAR(100), Price DECIMAL(10,2));
CREATE TABLE Orders (OrderID INT PRIMARY KEY, UserID INT, OrderDate DATE);
CREATE TABLE OrderDetails (
    OrderDetailID INT PRIMARY KEY, OrderID INT, ProductID INT, Quantity INT, UnitPrice DECIMAL(10,2)
);

-- Data Insertion
INSERT INTO Users VALUES (10, 'Rahul');
INSERT INTO Products VALUES (101, 'Laptop', 50000);
INSERT INTO Orders VALUES (500, 10, '2026-03-10');
INSERT INTO OrderDetails VALUES (1, 500, 101, 1, 50000);

-- Output Query (Revenue)
SELECT SUM(Quantity * UnitPrice) AS Total_Revenue FROM OrderDetails;
```

STDIN

Input for the program (Optional)

Output:

```
+---------------+
| Total_Revenue |
+---------------+
|      50000.00 |
+---------------+
```

Task 5 – University Examination Database

Design a database schema for a University Examination System and generate SQL queries using AI.

Instructions:

- Tables: Students, Subjects, Exams, Results.
- Define primary keys, foreign keys, and referential integrity constraints.
- **Generate queries:**
  - o Insert examination results for a student.
  - o Retrieve subject-wise marks for a student.
  - o Calculate average marks for each subject.

Expected Output:

- Normalized schema and SQL queries involving aggregation and joins.

AI Prompt: Create a schema for a University Exam System with Students, Subjects, Exams, and Results. Define referential integrity. Generate queries for inserting results, student subject marks, and subject averages.

Goal: Referential integrity for exam results.

queries.sql + 44hctxwwq

```sql
-- 1. PREREQUISITE TABLES (Required for Foreign Keys)
CREATE TABLE Students (
    StudentID INT PRIMARY KEY,
    StudentName VARCHAR(100)
);

CREATE TABLE Subjects (
    SubjectID INT PRIMARY KEY,
    SubjectName VARCHAR(100)
);

-- 2. SCHEMA DESIGN
CREATE TABLE Results (
    ResultID INT PRIMARY KEY AUTO_INCREMENT,
    StudentID INT,
    SubjectID INT,
    Marks INT,
    -- Establishing Referential Integrity
    FOREIGN KEY (StudentID) REFERENCES Students(StudentID) ON DELETE CASCADE,
    FOREIGN KEY (SubjectID) REFERENCES Subjects(SubjectID) ON DELETE CASCADE
);

-- 3. INSERT SAMPLE DATA (To ensure there is an output)
INSERT INTO Students VALUES (23001, 'Aditya Kumar');
INSERT INTO Subjects VALUES (101, 'Mathematics');
INSERT INTO Results (StudentID, SubjectID, Marks) VALUES (23001, 101, 88);

-- 4. GENERATE OUTPUT QUERIES
-- Query A: Retrieve subject-wise marks for a student
SELECT s.StudentName, sub.SubjectName, r.Marks
FROM Results r
JOIN Students s ON r.StudentID = s.StudentID
JOIN Subjects sub ON r.SubjectID = sub.SubjectID
WHERE r.StudentID = 23001;

-- Query B: Calculate average marks for each subject
SELECT sub.SubjectName, AVG(r.Marks) AS Average_Marks
FROM Results r
JOIN Subjects sub ON r.SubjectID = sub.SubjectID
GROUP BY sub.SubjectName;
```

STDIN

Input for the program (Optional)

Output:

```
+--------------+-------------+-------+
| StudentName  | SubjectName | Marks |
+--------------+-------------+-------+
| Aditya Kumar | Mathematics |    88 |
+--------------+-------------+-------+
+-------------+---------------+
| SubjectName | Average_Marks |
+-------------+---------------+
| Mathematics |       88.0000 |
+-------------+---------------+
```